# Reorder

## Reorder 난독화 분석

사전 지식

Smali 코드는 Obfuscapk의 **'작업 디렉터리(working directory)'** 안에 생성된다

1. **원본 `.apk` 파일:** 이 안에는 `classes.dex`라는 이름의, 컴파일된 **DEX 바이트코드**가 들어있다. (기계가 읽는 코드)
2. **디컴파일 단계:** Obfuscapk가 `apktool`을 사용해 이 `.apk` 파일을 디컴파일한다.
3. **작업 디렉터리 생성:** 이 과정에서 임시 파일들을 저장할 '작업 디렉터리'가 생긴다.
4. **`smali` 폴더 생성:**
   - 작업 디렉터리 안에 `smali`라는 폴더가 생긴다.
   - (앱이 커서 멀티덱스(multi-dex)라면 `smali_classes2`, `smali_classes3` 폴더들도 함께 생성된다.)

**결론:** `Reorder` 플러그인이 읽고, 수정하고, 뒤섞는 `.smali` 파일들은 모두 Obfuscapk의 **작업 디렉터리 내 `smali/` 폴더** 안에 들어있다.

## Reorder 난독화의 전체 메커니즘

- Smail 코드의 논리적 순서를 물리적으로 뒤섞어서, 리버스 엔지니어가 코드를 순서대로 읽어 내려가며 분석하는 것을 불가능하게 만든다

예시> 책을 읽는다고 가정하면
원본: 1page→ 2page→3page
Reorder: 1page→ 38page→4page

- 이 메커니즘은 Two-pass(두번의 작업)으로 수행된다

1. Pass 1: 준비 단계(코드 분할 및 제어 흐름 수정)

첫 번째로 파일을 읽으면서, 코드를 뒤섞을 '준비' 작업

- 메서드 스켄: Smail 파일의 메서드(.method)를 찾는다
- 안전지대 확인: try catch 블록 안에 있는 코드는 민감하므로 절대 건들지 않는다

- 블록 표식 확인: try catch 바깥의 안전한 코드를 만나면, 이 코드를 하나의 블록으로 만들기 위해 표식 삽입
  `goto/32 :L_RANDOM_1` (새 레이블로 무조건 점프)
  `#!code_block!#` (여기가 블록의 시작이라는 '마커')
  `:L_RANDOM_1` (점프가 도착할 '레이블')

- if 조건 뒤집기: 만약 코드가 if 구문이라면, 조건을 뒤집는다(리버싱을 어렵게 하기 위해서)
  -원본: `if-eq v0, v1, :LABEL_A` (v0과 v1이 **같으면** LABEL_A로 점프)

  -수정본:
  `if-ne v0, v1, :LABEL_B` (v0과 v1이 **다르면** LABEL_B로 점프)
  `goto/32 :LABEL_A` (같으면 원래대로 LABEL_A로 감)
  `:LABEL_B` (다르면 여기로 옴)

  ⇒ PASS1 결과: 원본 Smail 코드는 goto, if의 반대조건, #_code_block_1 마커가 붙게 됨

2. PASS 2: 수집 및 재배치(실제 뒤섞기)

- **블록 수집:** `.method`를 다시 스캔
- Pass 1에서 심어둔 `#!code_block!#` 마커를 찾는다
- 마커부터 다음 마커 전까지의 모든 Smali 코드를 하나의 `CodeBlock` 객체에 저장한다
- 메서드가 끝나는 `.end method`를 만날 때까지 이 `CodeBlock`들을 리스트(`code_blocks`)에 계속 추가한다
- **난독화 핵심:** `.end method`를 만나면, `random.shuffle(code_blocks)`를 호출한다
- **순서가 완전히 뒤섞인** `code_blocks` 리스트의 내용을 파일에 순서대로 다시 쓴다

⇒ 매서드 내의 코드 블록 순서가 완전히 물리적으로 뒤섞인다 but pass1에서 심어둔 `goto` 구문들이 '내비게이션' 역할을 하여, 실제 프로그램 실행(로직)은 원본과 동일하게 유지된다.

# 고도화 진행

Reorder 플러그인은 new_alignment와 달리 Smail 코드를 직접 파싱하고 수정하기 때문에 얼마나 안전하게 Smail 코드를 파싱하고 수정할지가 중요

## SmaliPrasingError 예외 처리 진행

Reorder 플러그인은 정규식(Regex) 파싱에 실패하면 AttributeError 를 발생시킨다 but 어떤 플러그인이 어떤 파일을 파싱하다 어떤 실패를 했는지는 알 수 없다

⇒ reorder.py의 obfusate 메서드의 맨끝에 있는 try except 블록을 수정한다.

Obfuscapk > src > obfuscapk > obfuscators > reorder > reorder.py > Reorder > obfuscate

```python
#!/usr/bin/env python3

import logging
import random
import re
from typing import List

from obfuscapk import obfuscator_category
from obfuscapk import util
from obfuscapk.obfuscation import Obfuscation
from obfuscapk.exceptions import PluginExecutionError,SmaliParsingError

```

```python
#[고도화 1] Smail 파싱 실패(AttributeError 등)로 인한 오류 처리
 except AttributeError as e:
     self.logger.error(
         'Error during execution of "{0}" obfuscator: Smail parsing failed: {1}'.format(
             self.__class__.__name__, e
         )
     )
     #AttributeError는 SmailParsingError로 래핑해서 상위 모듈에 전달
     raise SmaliParsingError(
         plugin_name=self.__class__.__name__,
         file_path=smali_file,
         details=str(e)
     ) from e



#[고도화 2] 그 외에 모든 일반적인 오류
 except Exception as e:
     self.logger.error(
         'Error during execution of "{0}" obfuscator: {1}'.format(
             self.__class__.__name__, e
         )
     )
     raise PluginExecutionError(
         plugin_name=self.__class__.__name__,
         details=str(e)
     ) from e

 finally:
     obfuscation_info.used_obfuscators.append(self.__class__.__name__)
```

exceptions.py 에 SmailParsingError 추가

```python
#SmailParsing이 실패했을 때(reorder 플러그인 등에서 사용)
#바로 프로그램이 죽는데 이 예외 클래스를 사용해서 어디서 파싱이 실패했는지 smail 파일 상세 정보 제공
class SmaliParsingError(PluginError):
    """
    Smali 코드 파싱 중 오류가 발생했을 때 발생하는 예외 클래스
    예: Reorder 플러그인이 Smali 파일을 파싱하는 도중 문법 오류를 발견한 경우
    """

    def __init__(self, plugin_name: str, file_path: str, details: str):
        self.plugin_name = plugin_name
        self.file_path = file_path
        self.details = details
        super().__init__(plugin_name, f"Smali 파일 '{file_path}' 파싱 중 오류 발생: {details}")
```

```python
#2. 플러그인 관련 오류
class PluginError(ObfuscapkException):
    """난독화 플러그인 관련 오류를 나타내는 예외 클래스"""
    def __init__(self, plugin_name: str, message: str):
        self.plugin_name = plugin_name
        self.message = message
        super().__init__(f"Plugin '{plugin_name}' error: {message}")
```

```python
class ObfuscapkException(Exception):
    """Obfuscapk 관련 예외의 기본 클래스"""

    pass
```

⇒ Smail 파일을 파싱 중 어떤 파일에서 오류가 났는지 명확한 예외처리

---

## if 구문 정규식 파싱에 대한 오류 처리

기존 코드에서는 if_match가 None인지 확인하는 코드가 없다 ⇒
정규식에서 레지스터와 레이블 파싱을 못하면 프로그램이 멈춘다

⇒if_match 객체가 None이 아닌지 확인하는 작업이 필요하다

reorder.py의 def obufsacte 매소드 수정

```python
#현재 명령어가 if 구문인지 확인
new_if = self.if_mapping.get(op_code, None)
if new_if: #if 구문이라면
    if_match = if_pattern.match(line) #두번째 정규식으로 레지스터와 레이블 파싱
    #정규식으로 파싱을 실패하면 프로그램이 멈춤 => if_match객체가 None이 아닌 경우에만 실행하게 해야 함
```

```
#[고도화 3] 정규식 파싱 실패에 대한 오류 처리
if if_match:
    random_label_name = util.get_random_string(16)
    out_file.write( #if 구문 조건 뒤집기 및 점프 구문 삽입
        "\t{if_cond} {register}, "
        ":gl_{new_label}\n\n".format(
            if_cond=new_if,
            register=if_match.group("register"),
            new_label=random_label_name,
        )
    )
    out_file.write( #원래 점프 레이블로 이동하는 goto 구문 삽입
        "\tgoto/32 :{0}\n\n".format(
        if_match.group("goto_label")
        )
    )
    out_file.write(
        "\t:gl_{0}".format(random_label_name)
    )
else: #정규식 파싱 실패 시
    #프로그램을 죽이지 말고, 경고 로그 남기고 원본 라인을 쓴다
    self.logger.warning(
       f"Could not parse 'if' statement in file '{smali_file}'. "
       f"Skipping line: {line.strip()}"

    )
    out_file.write(line)
```

⇒ Reorder 플러그인이 if_pattern 정규식으로 이해하지 못하는 복잡한 Smali if문을 만나도 AttributeError로 죽지 않고,
해당 라인만 안전하게 건너뛰고(Skipping line) 난독화를 계속 진행해 안전성을 향상시킨다